```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    RocCurveDisplay,
    classification_report
)
```

```
print(data.head())
```

```
Empty DataFrame
Columns: [{"cells":[{"cell_type":"code", execution_count:1, metadata:{"execution":{"iopub.execute_input":"2021-03-24T06:33:2
Index: []

[0 rows x 608 columns]
```

```
data = pd.read_csv("diabetes.csv")

print("First 5 rows:")
print(data.head())
```

```
First 5 rows:
Empty DataFrame
Columns: [{"cells":[{"cell_type":"code", execution_count:1, metadata:{"execution":{"iopub.execute_input":"2021-03-24T06:33:2
Index: []

[0 rows x 608 columns]
```

```
print("\nDataset Information:")
print(data.info())

print("\nDataset Description:")
print(data.describe())
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 0 entries
Columns: 608 entries, {"cells":[{"cell_type":"code" to nbformat_minor:4}
dtypes: object(608)
memory usage: 132.0+ bytes
None

Dataset Description:
       {"cells":[{"cell_type":"code" execution_count:1  \
count                              0                0
unique                             0                0
top                              NaN              NaN
freq                             NaN              NaN

       metadata:{"execution":{"iopub.execute_input":"2021-03-24T06:33:28.628604Z"  \
count                                                  0
unique                                                 0
top                                                  NaN
freq                                                 NaN

       iopub.status.busy:"2021-03-24T06:33:28.627755Z"  \
count                                                0
unique                                               0
top                                                NaN
freq                                               NaN

       iopub.status.idle:"2021-03-24T06:33:28.630575Z"  \
count                                                0
unique                                               0
top                                                NaN
freq                                               NaN
```

```
       shell.execute_reply:"2021-03-24T06:33:28.630104Z"}  \
count                                                   0
unique                                                  0
top                                                   NaN
freq                                                  NaN

       papermill:{"duration":0.019633 end_time:"2021-03-24T06:33:28.630673"  \
count                                 0                                   0
unique                                0                                   0
top                                 NaN                                 NaN
freq                                NaN                                 NaN

       exception:false start_time:"2021-03-24T06:33:28.61104"  ...  \
count                0                                     0  ...
unique               0                                     0  ...
top                NaN                                   NaN  ...
freq               NaN                                   NaN  ...

       version:"2.1.0"} kaggle:{"accelerator":"gpu"  \
count                 0                          0
unique                0                          0
top                 NaN                        NaN
freq                NaN                        NaN
```

```
print(data.columns.tolist())
```

```
['{"cells":[{"cell_type":"code"', 'execution_count:1', 'metadata:{"execution":{"iopub.execute_input":"2021-03-24T06:33:28.62
```

```
print(type(data))
print(data.shape)
print(data.columns.tolist())
```

```
<class 'pandas.core.frame.DataFrame'>
(0, 608)
['{"cells":[{"cell_type":"code"', 'execution_count:1', 'metadata:{"execution":{"iopub.execute_input":"2021-03-24T06:33:28.62
```

```
cols = [
    'Glucose',
    'BloodPressure',
    'SkinThickness',
    'Insulin',
    'BMI'
]

available_cols = [col for col in cols if col in data.columns]
if len(available_cols) == len(cols):
    print(data[cols].eq(0).sum())
else:
    print("Error: The expected columns do not exist in the dataset.")
    print("Found columns instead:", data.columns.tolist()[:5])
```

```
Error: The expected columns do not exist in the dataset.
Found columns instead: ['{"cells":[{"cell_type":"code"', 'execution_count:1', 'metadata:{"execution":{"iopub.execute_input":
```

```
import pandas as pd
import numpy as np

# Download and load the genuine Pima Indians Diabetes dataset directly from a reliable online source
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"
column_names = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'A

try:
    data = pd.read_csv(url, names=column_names)
    print("Dataset successfully loaded from URL!")
except Exception as e:
    print("Error loading dataset:", e)

# Columns where 0 represents missing values
cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

# Replace 0s with NaN
data[cols] = data[cols].replace(0, np.nan)

print("\nFirst 5 rows:")
display(data.head())
print("\nMissing Values count per column:")
print(data.isnull().sum())
```

```
Dataset successfully loaded from URL!

First 5 rows:
```

| | Pregnancies | Glucose | BloodPressure | SkinThickness | Insulin | BMI | DiabetesPedigreeFunction | Age | Outcome |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 6 | 148.0 | 72.0 | 35.0 | NaN | 33.6 | 0.627 | 50 | 1 |
| **1** | 1 | 85.0 | 66.0 | 29.0 | NaN | 26.6 | 0.351 | 31 | 0 |
| **2** | 8 | 183.0 | 64.0 | NaN | NaN | 23.3 | 0.672 | 32 | 1 |
| **3** | 1 | 89.0 | 66.0 | 23.0 | 94.0 | 28.1 | 0.167 | 21 | 0 |
| **4** | 0 | 137.0 | 40.0 | 35.0 | 168.0 | 43.1 | 2.288 | 33 | 1 |

```
Missing Values count per column:
Pregnancies                 0
Glucose                     5
BloodPressure              35
SkinThickness             227
Insulin                   374
BMI                        11
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

```
X = data.drop("Outcome", axis=1)
y = data["Outcome"]

print("Input Features:")
print(X.head())

print("\nOutput:")
print(y.head())
```

```
Input Features:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6    148.0           72.0           35.0      NaN  33.6
1            1     85.0           66.0           29.0      NaN  26.6
2            8    183.0           64.0            NaN      NaN  23.3
3            1     89.0           66.0           23.0     94.0  28.1
4            0    137.0           40.0           35.0    168.0  43.1

   DiabetesPedigreeFunction  Age
0                     0.627   50
1                     0.351   31
2                     0.672   32
3                     0.167   21
4                     2.288   33

Output:
0    1
1    0
2    1
3    0
4    1
Name: Outcome, dtype: int64
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training data size:", X_train.shape)
print("Testing data size:", X_test.shape)
```

```
Training data size: (614, 8)
Testing data size: (154, 8)
```

```
imputer = SimpleImputer(strategy="median")

X_train = imputer.fit_transform(X_train)
X_test = imputer.transform(X_test)
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
model = LogisticRegression(
    class_weight="balanced",
    max_iter=1000
)

model.fit(X_train, y_train)

print("Model Training Completed!")
```

```
Model Training Completed!
```

```
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]
```

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_prob)

print("Accuracy :", round(accuracy, 3))
print("Precision:", round(precision, 3))
print("Recall   :", round(recall, 3))
print("F1-Score :", round(f1, 3))
print("ROC-AUC  :", round(roc_auc, 3))
```

```
Accuracy : 0.734
Precision: 0.603
Recall   : 0.704
F1-Score : 0.65
ROC-AUC  : 0.813
```

```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.82      0.75      0.79       100
           1       0.60      0.70      0.65        54

    accuracy                           0.73       154
   macro avg       0.71      0.73      0.72       154
weighted avg       0.75      0.73      0.74       154
```

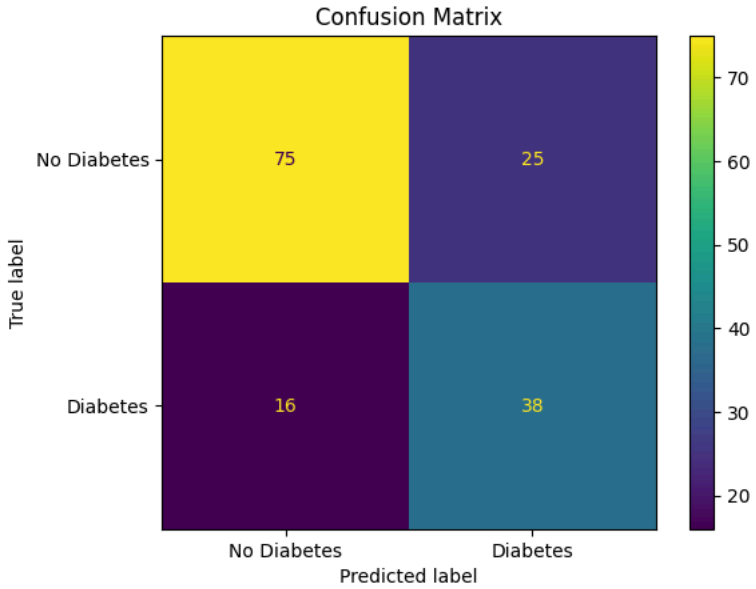
```
cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
print(cm)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No Diabetes", "Diabetes"]
)

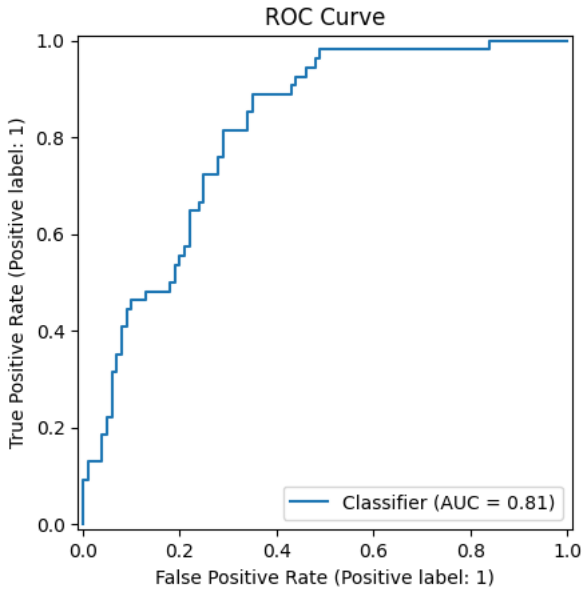
disp.plot()
plt.title("Confusion Matrix")
plt.show()
```

```
Confusion Matrix:
[[75 25]
 [16 38]]
```




```
RocCurveDisplay.from_predictions(y_test, y_prob)

plt.title("ROC Curve")
plt.show()
```




```
feature_names = X.columns

coefficients = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": model.coef_[0]
})

coefficients = coefficients.sort_values(
    by="Coefficient",
    ascending=False
)

print(coefficients)
```

```
                    Feature  Coefficient
1                   Glucose     1.183483
5                       BMI     0.709592
0               Pregnancies     0.372902
6  DiabetesPedigreeFunction     0.287734
7                       Age     0.186434
3             SkinThickness     0.013980
2             BloodPressure    -0.014612
4                   Insulin    -0.044609
```

```
plt.figure(figsize=(10, 6))

plt.bar(coefficients["Feature"], coefficients["Coefficient"])

plt.xlabel("Features")
plt.ylabel("Coefficient Value")
plt.title("Logistic Regression Feature Coefficients")

plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```

